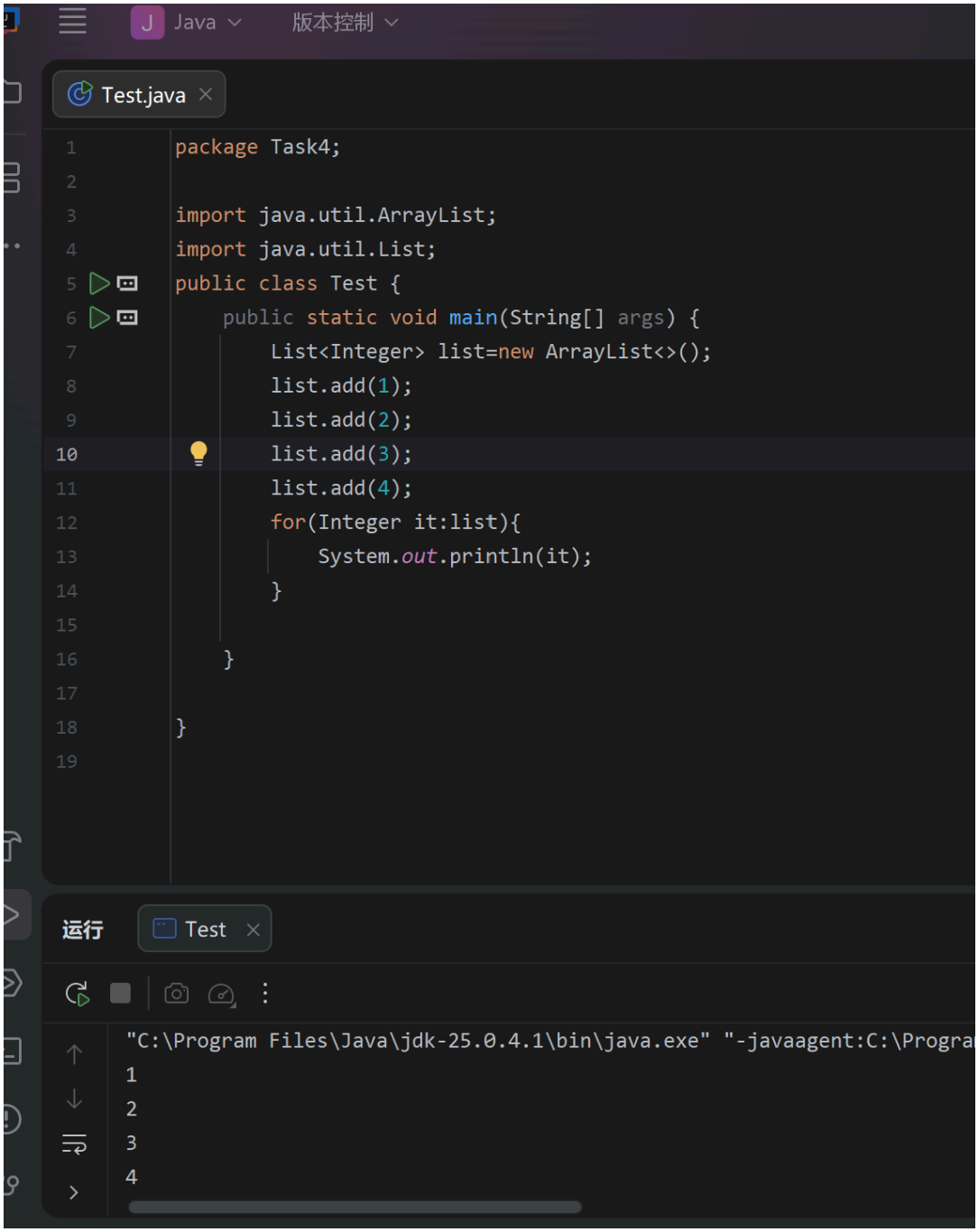
Task1

1. Collection定义了许多方法可以供实现类用；
List继承了Collection，增加了可以用下角标对集合修改的方法；
Set继承Collection,集合不重复；
Map定义许多方法可以给实现类用； ArrayList实现在List的基础上可以自动扩增；
LinkedList可以实现首尾的更改；
HashSet实现Set；
TreeSet 以升序排列
HashMap 存键值对，通过key查value，实现了Map；
TreeMap实现Map，排序，小到大。

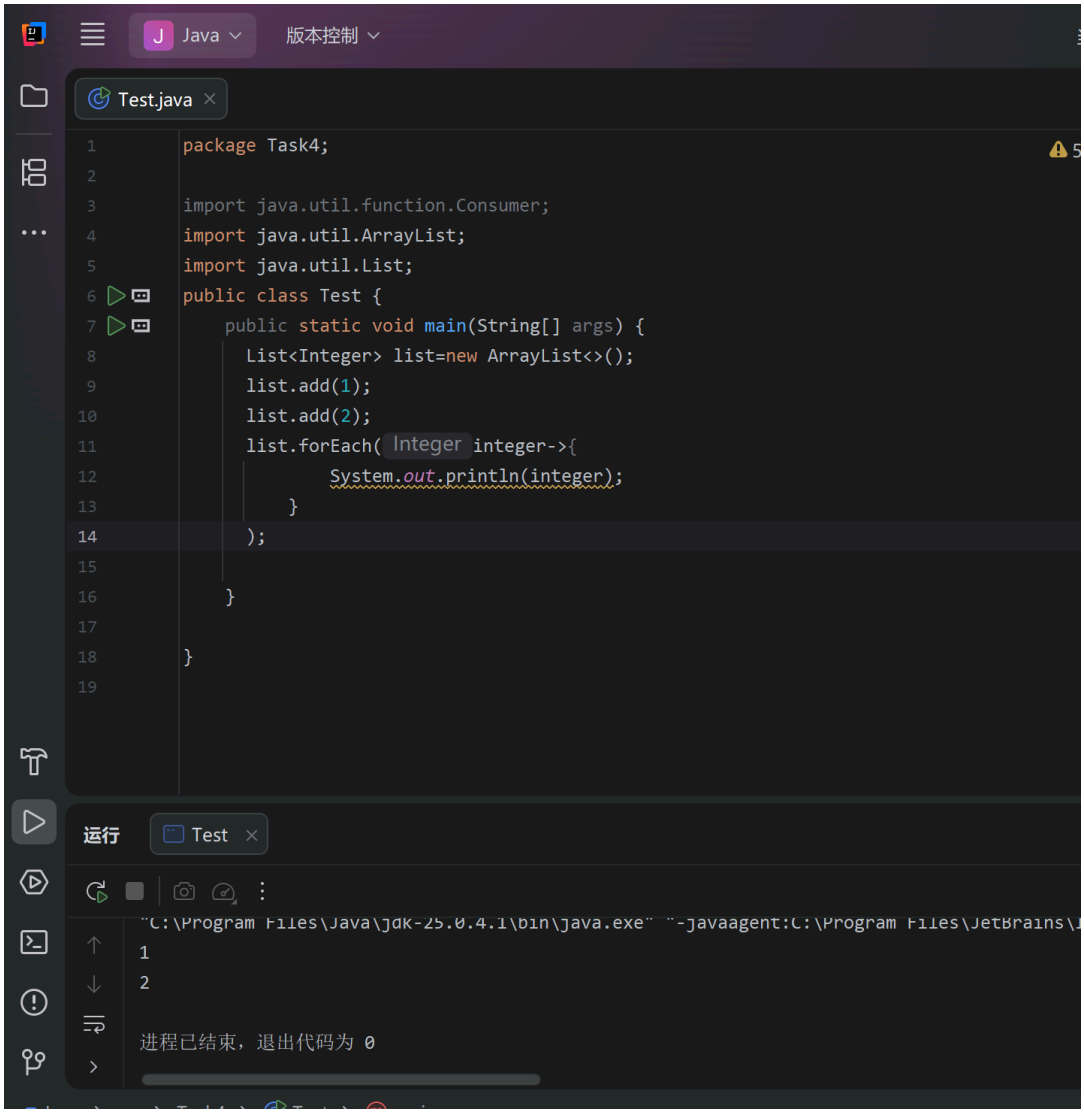
数组的长度定义的时候就固定了，但是集合不一定;
数组可以存对象和基本数据类型，但是集合却只能存对象；
集合里面有多种API，用起来更方便。
2.



3.(1)写在一个类里面的只使用一次的类，没有名字，实现父类或接口；

(2)函数式接口就是只有一个抽象方法的接口；

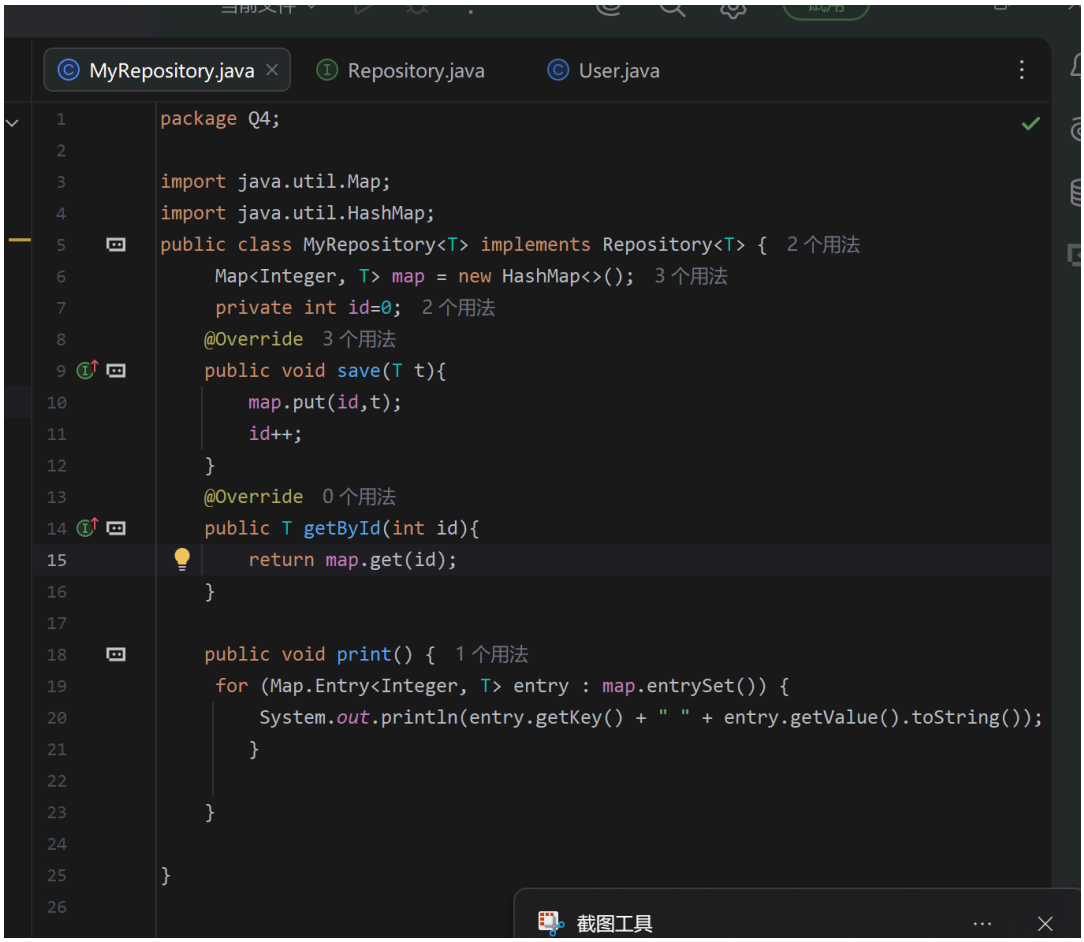
(3)



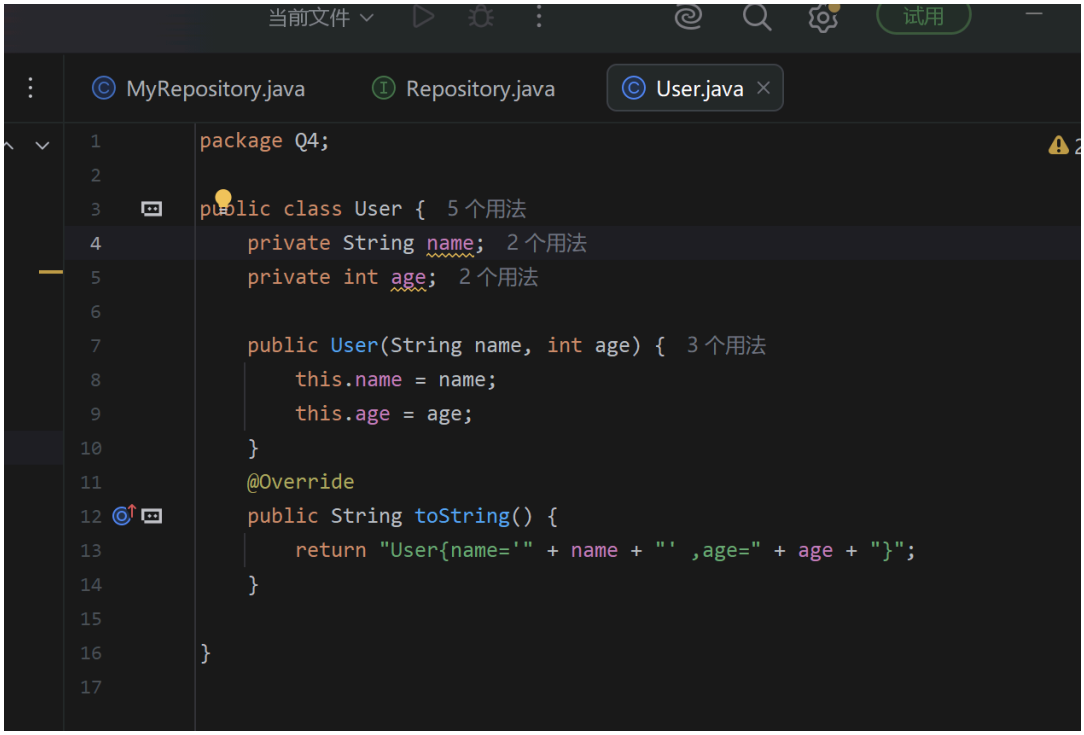
用法：就是表示一个函数式接口，前面写（形参），打一个"->"后面写重写的方法体

Task2

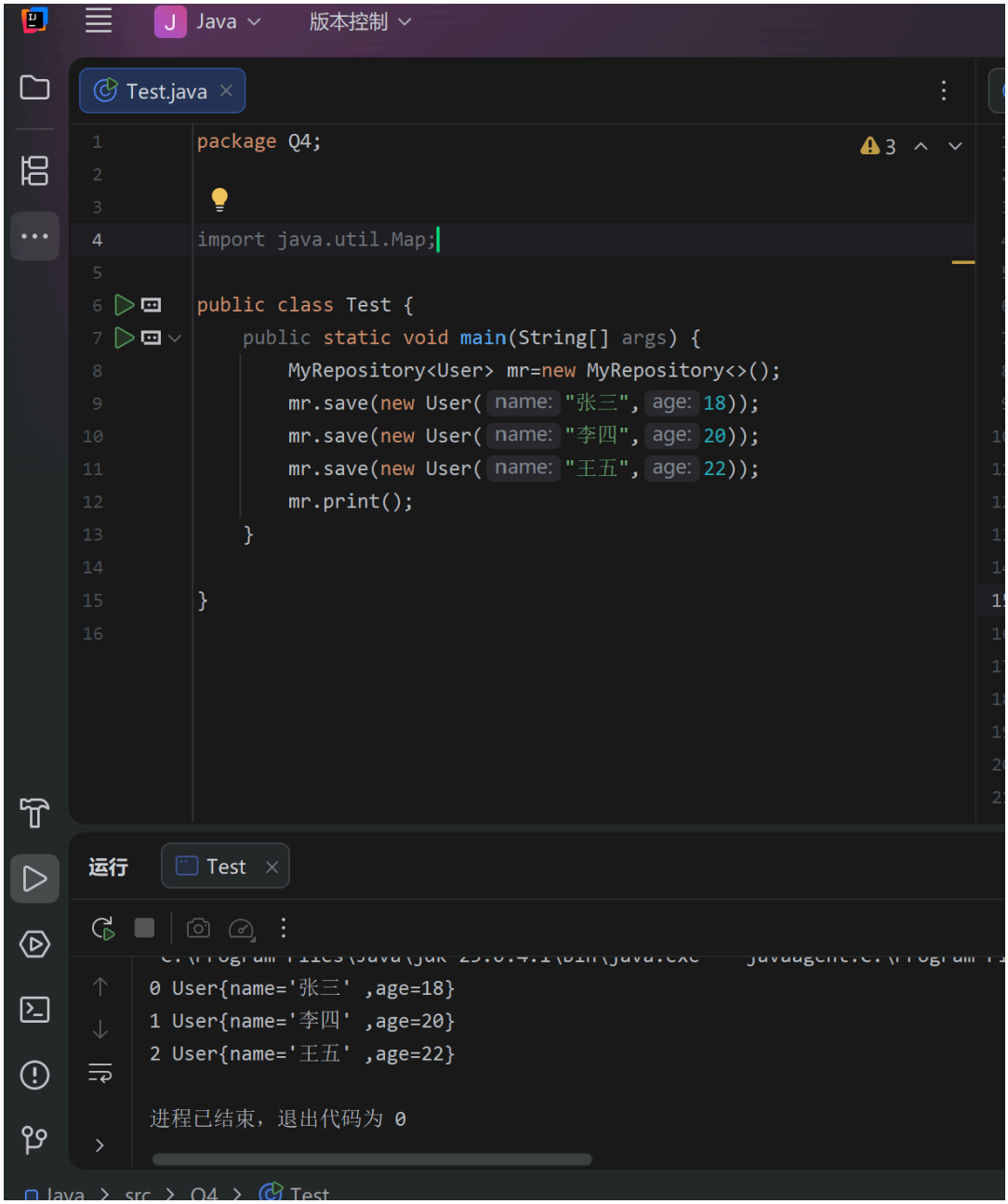
4.


MyRepository.java
Repository.java
User.java
package Q4;
import java.util.Map;
import java.util.HashMap;
public class MyRepository<T> implements Repository<T> { 2 个用法
Map<Integer, T> map = new HashMap<>(); 3 个用法
private int id=0; 2 个用法
@Override 3 个用法
public void save(T t){
map.put(id,t);
id++;
}
@Override 0 个用法
public T getById(int id){
return map.get(id);
}
public void print() { 1 个用法
for (Map.Entry<Integer, T> entry : map.entrySet()) {
System.out.println(entry.getKey() + " " + entry.getValue().toString());
}
}
}
截图工具

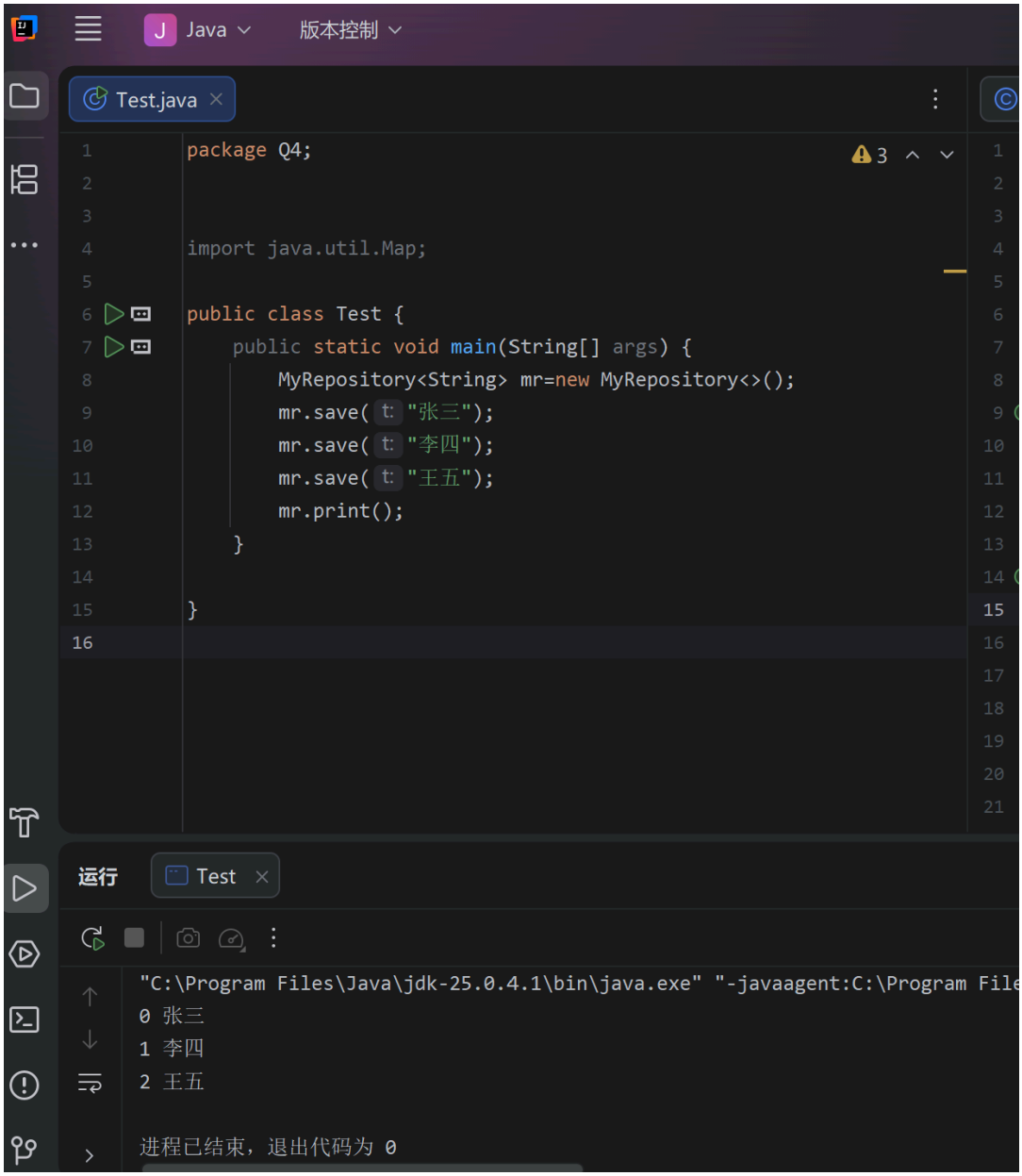

MyRepository.java
Repository.java
User.java
package Q4;
1
public interface Repository< T> { 1 个用法 1 个实现
T getById(int id); 0 个用法 1 个实现
void save(T t); 3 个用法
}


当前文件
试用
MyRepository.java
Repository.java
User.java
package Q4;
public class User { 5 个用法
private String name; 2 个用法
private int age; 2 个用法
public User(String name, int age) { 3 个用法
this.name = name;
this.age = age;
}
@Override
public String toString() {
return "User{name='" + name + "' ,age=" + age + "}";
}
}

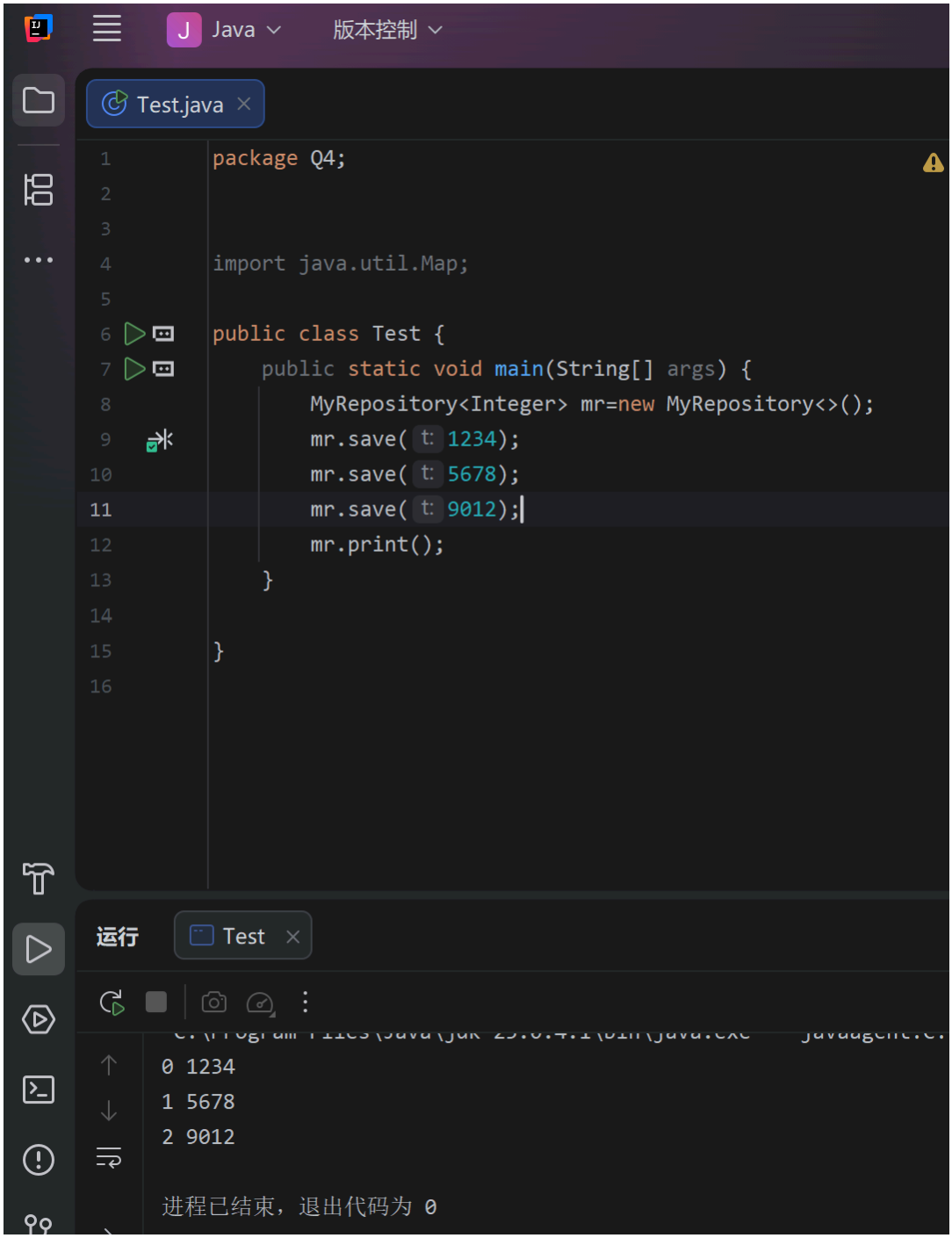
User：


Java
版本控制
Test.java
3
package Q4;
import java.util.Map;
public class Test {
public static void main(String[] args) {
MyRepository<User> mr=new MyRepository<>();
mr.save(new User(name: "张三", age: 18));
mr.save(new User(name: "李四", age: 20));
mr.save(new User(name: "王五", age: 22));
mr.print();
}
}
运行
Test
0 User{name='张三' ,age=18}
1 User{name='李四' ,age=20}
2 User{name='王五' ,age=22}
进程已结束，退出代码为 0
Java > src > Q4 > Test

String：

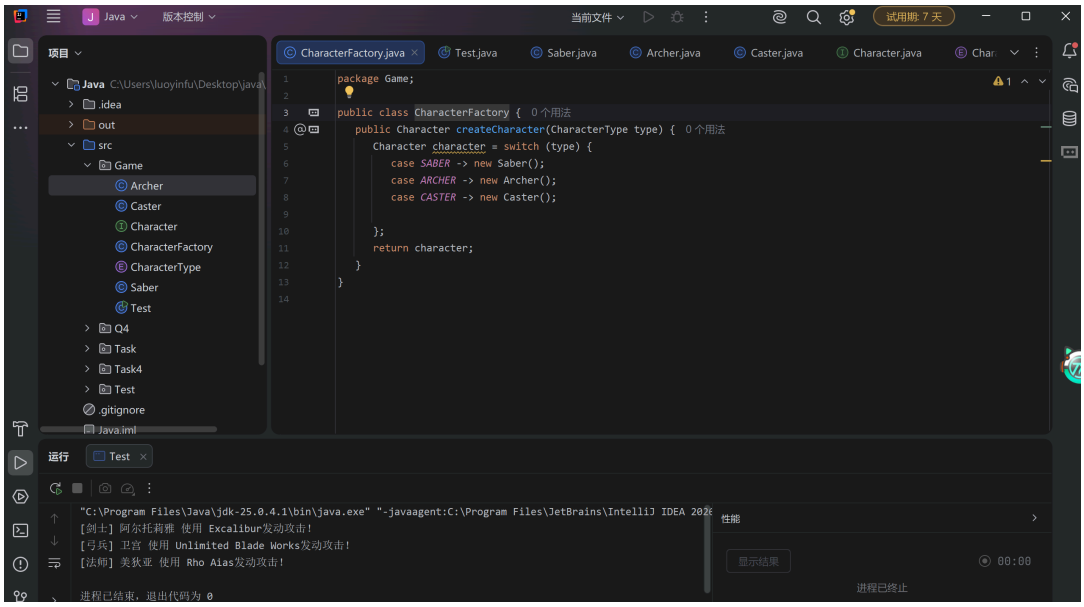

Java
版本控制
Test.java
package Q4;
import java.util.Map;
public class Test {
public static void main(String[] args) {
MyRepository<String> mr=new MyRepository<>();
mr.save(t: "张三");
mr.save(t: "李四");
mr.save(t: "王五");
mr.print();
}
}
运行
Test
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaagent:C:\Program File
0 张三
1 李四
2 王五
进程已结束，退出代码为 0

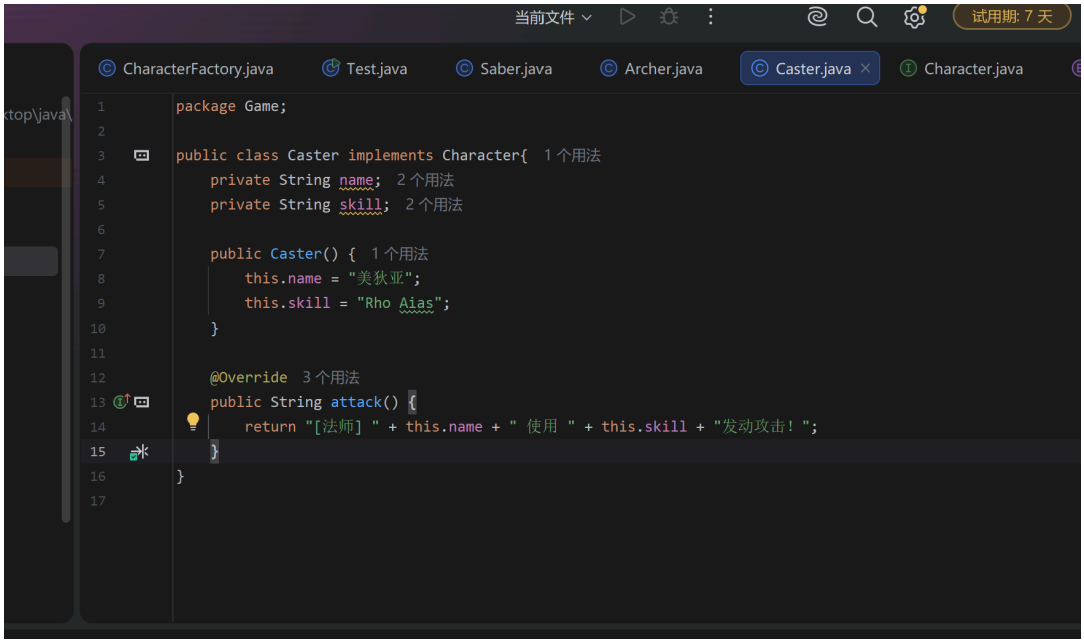
Integer：

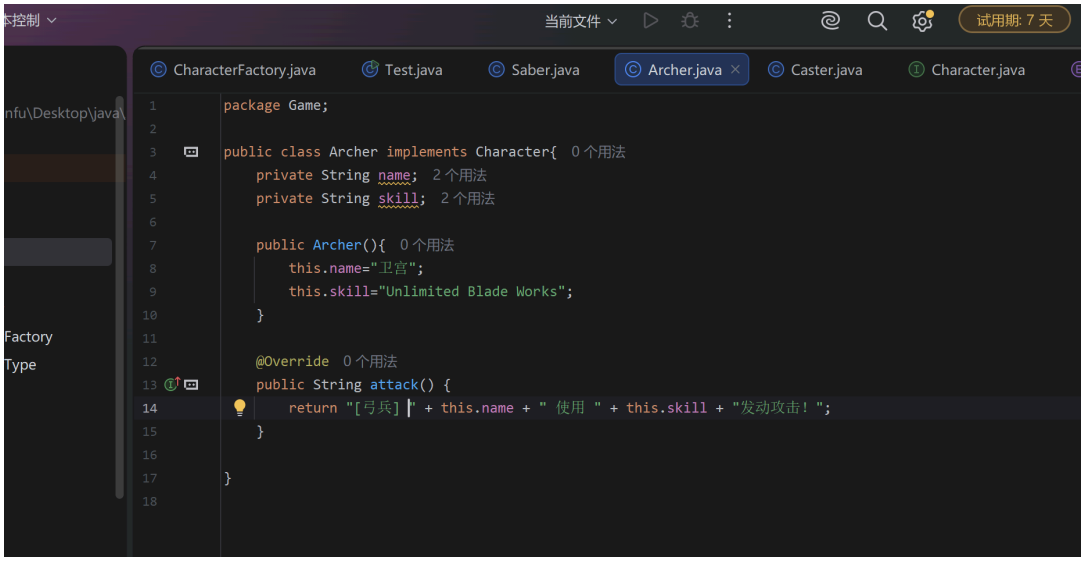

Java
版本控制
Test.java
package Q4;
import java.util.Map;
public class Test {
public static void main(String[] args) {
MyRepository<Integer> mr=new MyRepository<>();
mr.save(t: 1234);
mr.save(t: 5678);
mr.save(t: 9012);
mr.print();
}
}
运行
Test
0 1234
1 5678
2 9012
进程已结束，退出代码为 0

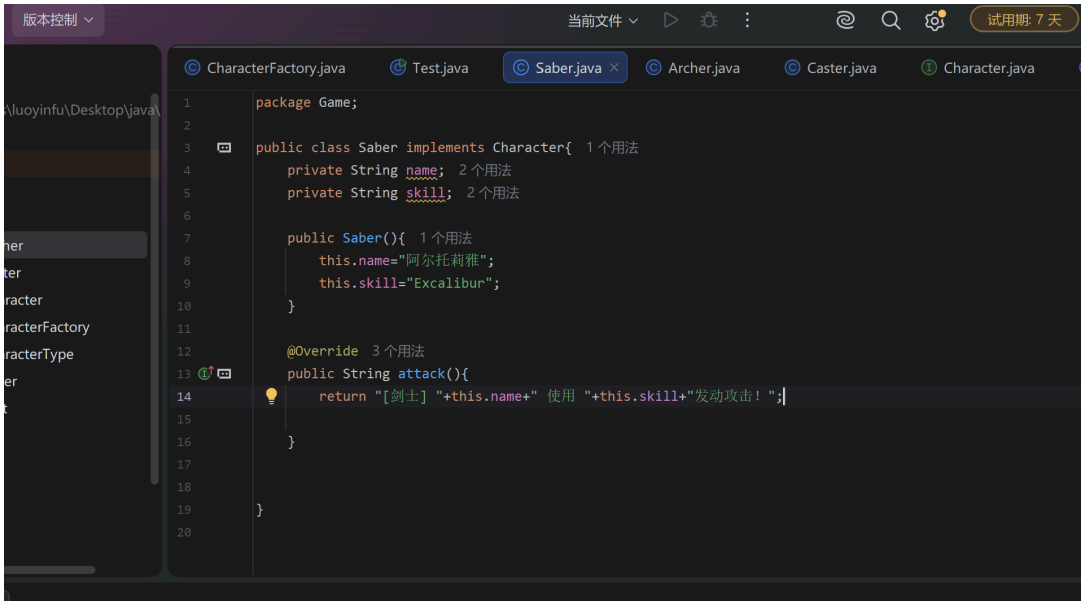
Task3

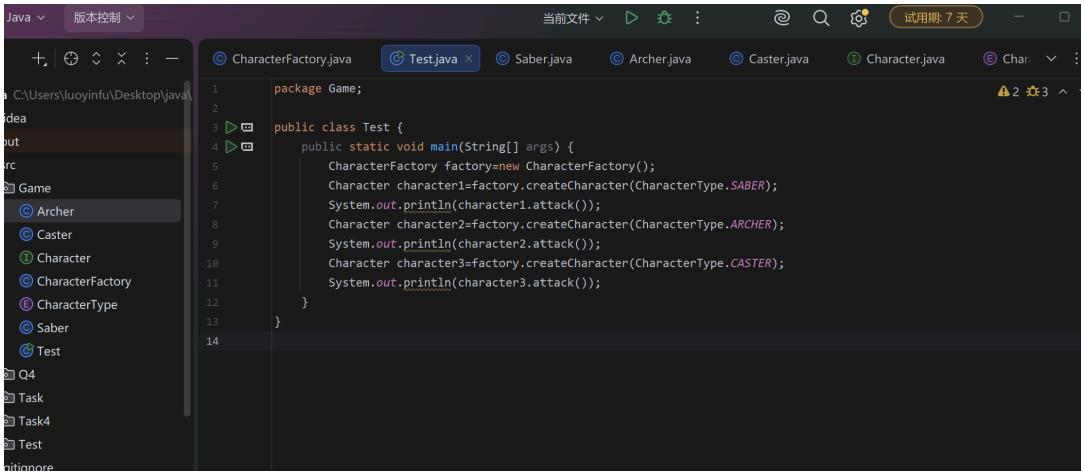
1.

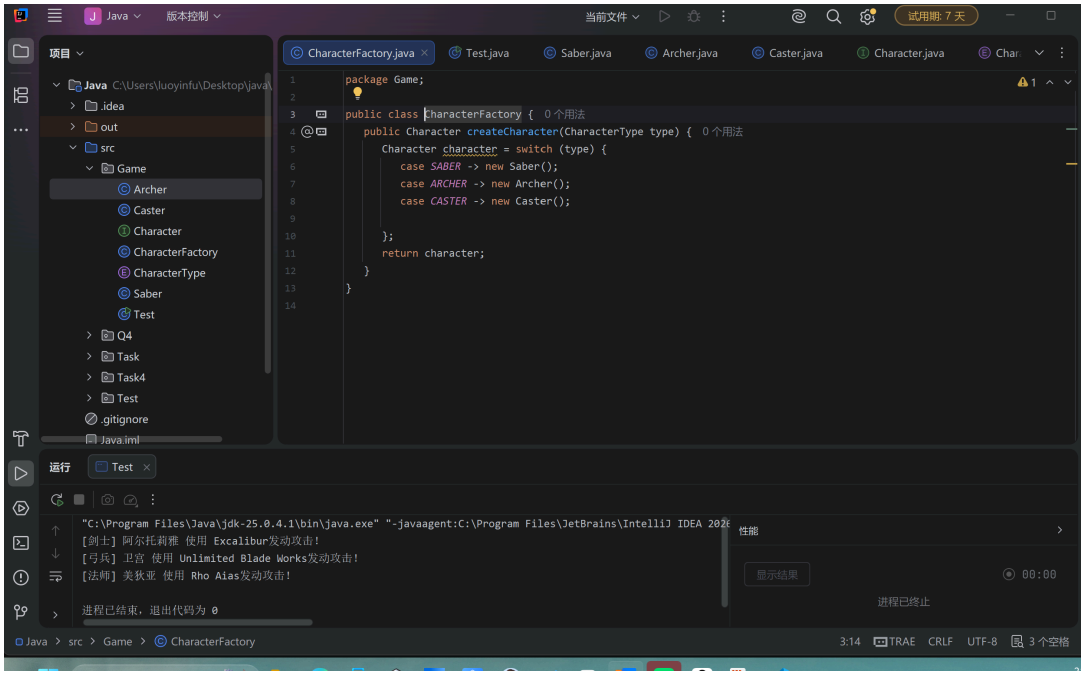

CharacterFactory.java
Test.java
Saber.java
Archer.java
Caster.java
Character.java
package Game;
public class CharacterFactory { 0 个用法
public Character createCharacter(CharacterType type) { 0 个用法
Character character = switch (type) {
case SABER -> new Saber();
case ARCHER -> new Archer();
case CASTER -> new Caster();
};
return character;
}
}
运行
Test
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 202
[剑士] 阿尔托莉雅 使用 Excalibur发动攻击!
[弓兵] 卫宫 使用 Unlimited Blade Works发动攻击!
[法师] 美狄亚 使用 Rho Aias发动攻击!
进程已结束，退出代码为 0

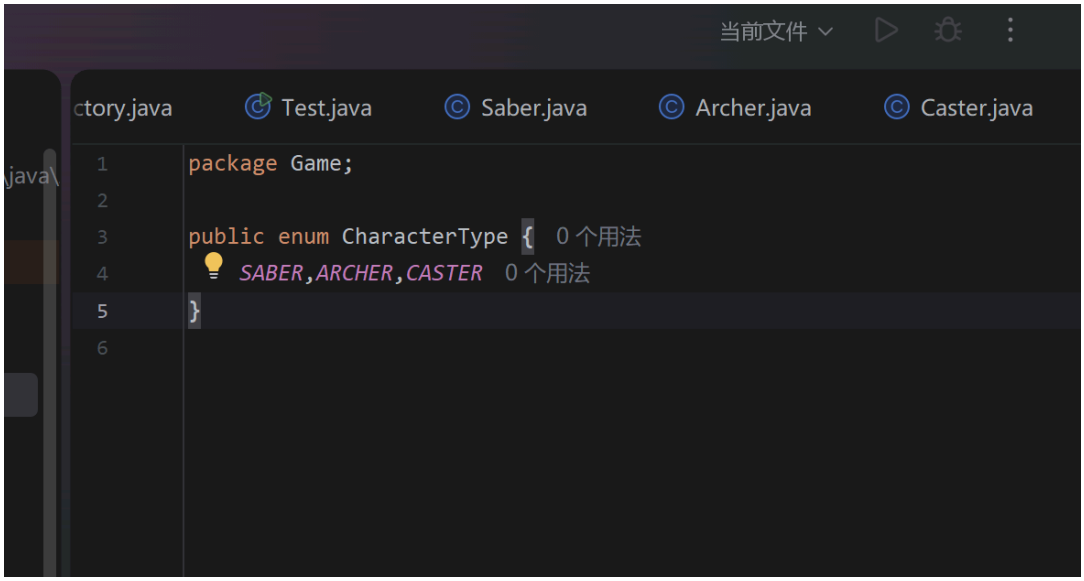

当前文件
试用期: 7 天
CharacterFactory.java
Test.java
Saber.java
Archer.java
Caster.java
Character.java
package Game;
public class Caster implements Character{ 1 个用法
private String name; 2 个用法
private String skill; 2 个用法
public Caster() { 1 个用法
this.name = "美狄亚";
this.skill = "Rho Aias";
}
@Override 3 个用法
public String attack() {
return "[法师] " + this.name + " 使用 " + this.skill + "发动攻击! ";
}
}


当前文件
试用期: 7 天
CharacterFactory.java
Test.java
Saber.java
Archer.java
Caster.java
Character.java
package Game;
public class Archer implements Character{ 0 个用法
private String name; 2 个用法
private String skill; 2 个用法
public Archer(){ 0 个用法
this.name="卫宫";
this.skill="Unlimited Blade Works";
}
@Override 0 个用法
public String attack() {
return "[弓兵] " + this.name + " 使用 " + this.skill + "发动攻击! ";
}
}


版本控制
当前文件
试用期: 7 天
CharacterFactory.java
Test.java
Saber.java
Archer.java
Caster.java
Character.java
package Game;
public class Saber implements Character{ 1 个用法
private String name; 2 个用法
private String skill; 2 个用法
public Saber(){ 1 个用法
this.name="阿尔托莉雅";
this.skill="Excalibur";
@Override 3 个用法
public String attack(){
return "[剑士] "+this.name+" 使用 "+this.skill+"发动攻击！";


Java
版本控制
当前文件
试用期: 7 天
CharacterFactory.java
Test.java
Saber.java
Archer.java
Caster.java
Character.java
Game
Archer
Caster
Character
CharacterFactory
CharacterType
Saber
Test
Q4
Task
Task4
Test
package Game;
public class Test {
public static void main(String[] args) {
CharacterFactory factory=new CharacterFactory();
Character character1=factory.createCharacter(CharacterType.SABER);
System.out.println(character1.attack());
Character character2=factory.createCharacter(CharacterType.ARCHER);
System.out.println(character2.attack());
Character character3=factory.createCharacter(CharacterType.CASTER);
System.out.println(character3.attack());





Mind

1.对于复杂任务的解决，要理清思路，搞清各个类之间的关系，并用合理的关系(接口继承)来表述；

2对于相似的概念，可以找之间的关系记忆，HashSet与HashMap，TreeSet和TreeMap；